

Nithi Sarathi

An AI-powered conversational assistant that helps Indian citizens discover government welfare schemes they may be eligible for.

The Problem

India has 3,000+ government welfare schemes across central and state levels — for farmers, students, gig workers, women, senior citizens, and more. Most citizens never discover the schemes they qualify for, because:

- You need to already know a scheme's name to search for it
- Eligibility rules are written in dense legal/bureaucratic language
- There's no single place that asks "who are you?" and tells you what applies to you

Manually researching this today takes a citizen 2–3 hours of scattered searching, if they even know where to start. NitiSarathi turns that into a few minutes of natural conversation.

The Solution

NitiSarathi is a chat assistant that:

- Has a natural, adaptive conversation with the citizen — asking follow-up questions based on what they've already shared (no fixed script/form)
- Uses Retrieval-Augmented Generation (RAG) to pull only the most relevant schemes from a knowledge base of 3,000+ schemes, based on the meaning of what the citizen says
- Uses Gemma 4 to reason over the retrieved scheme text and explain, in plain language, why the citizen is likely eligible, possibly eligible, or should check further
- Returns a structured, easy-to-scan eligibility breakdown — not a wall of text

For the project purpose only 12 policies are used. However, this can be scaled to even 3000+ policies.

RAG — why it's needed here

Stuffing all 3,000+ scheme descriptions into every prompt is slow, expensive, and eventually impossible as the token limit is exceeded. Instead:

- Each scheme is stored as a numerical embedding (its "meaning fingerprint")
- When a citizen describes their situation, that message is embedded too
- The system retrieves only the schemes whose meaning is closest to the citizen's message — concept matching, not exact keyword matching

Important nuance: RAG only finds candidates. It does not decide eligibility by itself — Gemma 4 still has to read the retrieved scheme text and reason about whether this specific citizen's situation actually fits. Two AI-dependent steps, not one, both running on the Gemini API.

Flow of the model

Stage	Input Artifact	Processing Component	Output / Purpose
1. Data Ingestion	data/schemes-list.csv	n8n-workflows/NitiSarathi-Ingestion.json	Structured Knowledge Base
2. Reasoning & Logic	prompts/system-prompt.md	n8n-workflows/NitiSarathi-Backend.json	Gemma 4 Query Processing
3. Demonstration	Completed Pipeline	demo/demo-video-link.md	End-to-End Walkthrough

Procedure:

- Import n8n-workflows/NitiSarathi-Ingestion.json into your own n8n instance and run it once to embed the scheme data from data/schemes-list.csv
- Import n8n-workflows/NitiSarathi-Backend.json, add your own Google AI Studio API key, and activate the workflow
- Copy the resulting webhook URL into the Emergent frontend prompt (see prompts/ for the exact prompt used)
- Open the deployed Emergent URL and start chatting

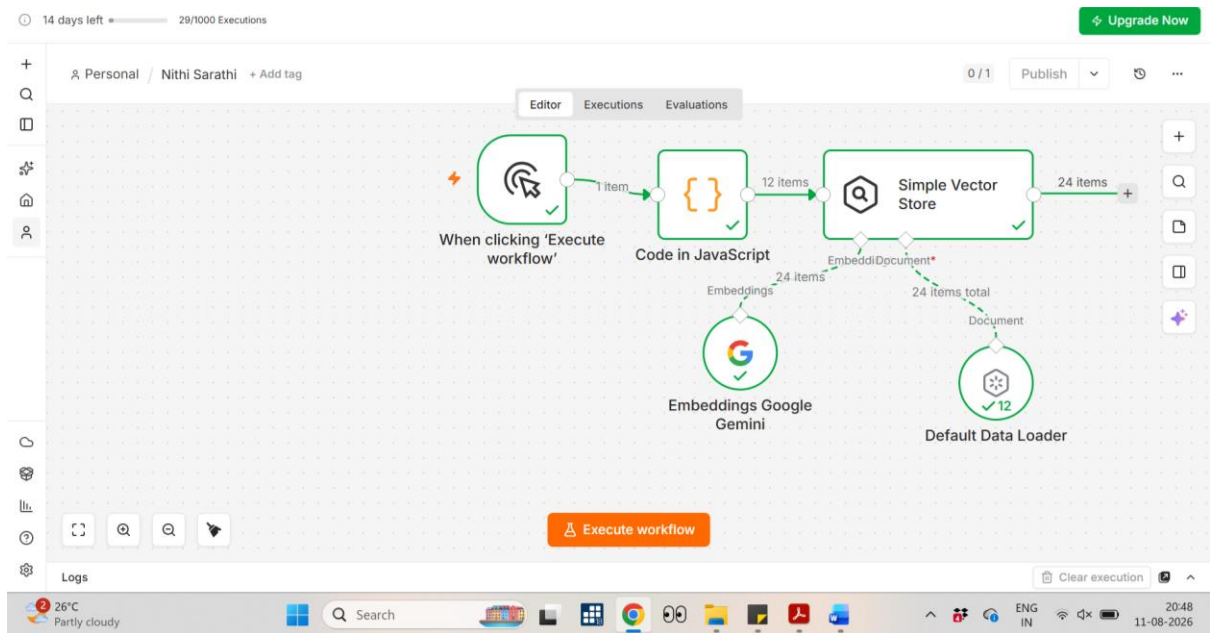
Benefit of Model:

- Who it helps most: rural households, gig workers, first-time scheme applicants, and community health workers who currently do this discovery manually for others
- Time saved: ~2–3 hours of manual research per person → a few minutes of conversation
- Scale: the same architecture generalizes to any country/domain with a large, fragmented catalog of eligibility-based programs

Future scope:

- Add regional language support
- Expand the scheme corpus beyond the initial ~12–20 used for the hackathon demo to the full 3,000+
- Add a feedback loop so citizens can confirm/deny actual eligibility outcomes, improving future retrieval and reasoning
- Persist conversation history per citizen (currently session-only)

Front end



This screenshot shows the configuration and output of the 'Code in JavaScript' node. The left panel shows the input and variables. The middle panel shows the JavaScript code. The right panel shows the output as a table.

Code in JavaScript Configuration:

- Language:** JavaScript
- JavaScript Code:**

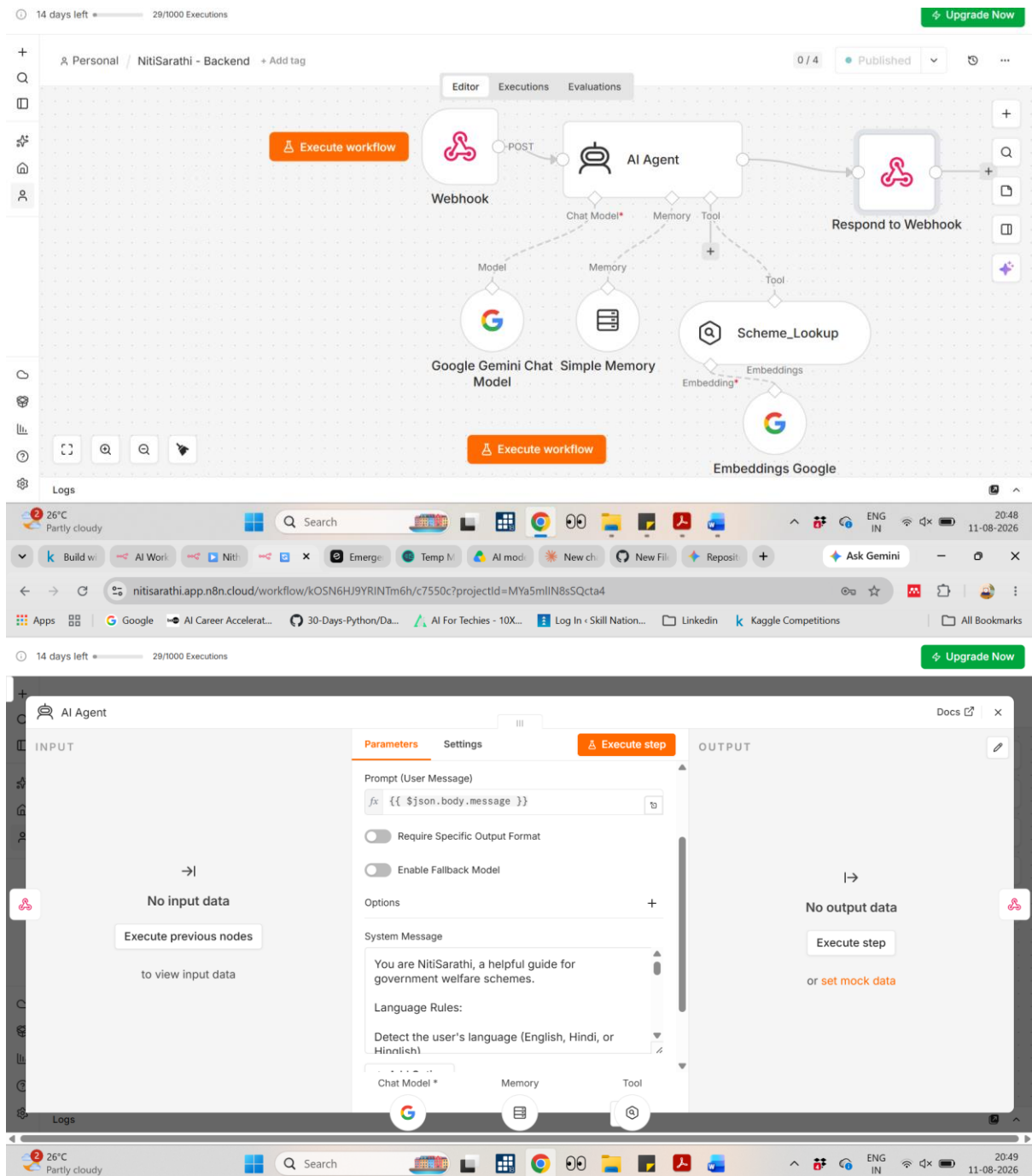
```
1 const schemes = [{}];
15 return schemes.map(s => ({ json: s }));
```

Output (12 Items):

name	text
Ayushman Bharat - PMJAY	records. Excludes institutional landholders, constitutional post holders, serving/retired government employees (except Class IV/Group D), pensioners drawing Rs 10,000+/month, income-tax payers, and professionals (doctors, engineers, CAs, lawyers).
PM Awas Yojana	Ayushman Bharat - PMJAY: Cashless health cover of Rs 5 lakh per family per year (floater, no cap on family size) for secondary/tertiary hospitalization, no premium, no waiting period -- pre-existing conditions covered from day one. BPL eligibility is via SECC 2011 deprivation/occupational criteria (landless labourers, kutcha housing, SC/ST households, disabled with no able-bodied adult, etc). All citizens 70+ are covered regardless of income under the Ayushman Vay Vandana Card, with an extra Rs 5 lakh cover.
PM Awas Yojana	PM Awas Yojana (Urban and Gramin): Housing assistance for families without a pucca house. PMAY-Gramin gives Rs 1.20 lakh (plain areas) or Rs 1.30 lakh (hilly) to SECC/Awas+ listed households with monthly income up to Rs 15,000, prioritizing SC/ST, widows, and

Gemma 4 System Prompt — Workflow B (Backend)

This is the system prompt used to drive Gemma 4's conversation, tool-calling, and eligibility reasoning in the n8n Backend workflow.



Emergent Prompt

Build a simple, clean chat interface web app called "NitiSarathi" — a conversational assistant that helps Indian citizens discover government welfare schemes they may be eligible for.

1. UI: A single-page chat interface (like a messaging app). Show a welcome message: "Hi, I'm NitiSarathi. Tell me a bit about yourself — your occupation, location, or situation — and I'll help you find government schemes you may qualify for." Include a text input box, a send button, and a scrolling message thread (user messages right-aligned, assistant messages left-aligned).

2. Backend logic: The frontend has NO built-in logic, no hardcoded responses, and no local AI calls. Every single user message must be sent via POST request to this webhook endpoint:

https://nitisarathi.app.n8n.cloud/webhook/Nithi_Sarathi

Send the request as JSON with this shape:

```
{
  "message": "<the user's typed text>",
  "session_id": "<a persistent random ID generated once per browser session>"
}
```

3. Session continuity: Generate a session_id (UUID) when the chat first loads and reuse the same session_id for every message in that browser session, so the backend can maintain conversation context.

4. Handling the response — TWO POSSIBLE SHAPES:

The backend may return either:

(a) A plain string reply — render it as a normal chat bubble, left-aligned, like standard conversational text. Use this during the back-and-forth phase, e.g. before enough information has been gathered to suggest schemes.

(b) A structured JSON object, shaped like this:

```
{
  "likely_eligible": [ { "scheme": "...", "reason": "...", "next_step": "..." }, ... ],
  "worth_checking": [ { "scheme": "...", "uncertainty": "..." }, ... ],
  "disclaimer": "..."
}
```

When you receive shape (b), do NOT render it as a plain chat bubble. Instead render a structured results card:

- For each item in likely_eligible: a card with a green left border. Scheme name as a bold heading, the reason as body text below it, and the next_step as a bolded action line at the bottom.
- For each item in worth_checking: a card with a yellow/amber left border, scheme name as heading, and the uncertainty text as body text.
- Below all the cards, show the disclaimer text in small italic gray font.
- Stack likely_eligible cards above worth_checking cards. If either array is empty, don't render that section's heading.
- Keep card styling visually consistent with the rest of the chat interface (same font, spacing, rounded corners as chat bubbles).

Detect which shape you received by checking whether the response is valid JSON containing the keys likely_eligible / worth_checking / disclaimer. If it's plain text or doesn't match that shape, fall back to a normal chat bubble.

5. Show a "typing..." indicator while waiting for the webhook response.

6. Error handling: If the webhook call fails or times out, show a friendly message like "Sorry, I couldn't reach the server. Please try again in a moment." Do not crash the UI.

7. Styling: Clean, minimal, mobile-friendly, warm and simple. White background, one accent color (e.g. green or blue) for the send button and user bubbles. No unnecessary branding.

8. Deploy this as a live, publicly accessible web app (no login required) so it can be used as a demo link.

Do not add any AI/LLM logic on the frontend itself — Emergent's only job is to be the chat window, manage `session_id`, call the webhook above, and render either plain bubbles or structured eligibility cards depending on the response shape. All intelligence lives in the n8n backend.